Royaume du Maroc
Université Sultan Moulay Slimane
École Nationale des Sciences Appliquées
Béni Mellal
Transformation Digitale Industrielle

Spécialité : Transformation Digitale Industrielle
Module : Génie Logiciel

TP N° 2 : Docker

Filière : **TDI**
Niveau : **2ème CI**

Effectuée par :

HAMMAOUI Fatima Ezzahra

Encadré(e) par :

Pr. BE.ELBAGHAZAOUI

Objectif du TP :

Ce TP a pour objectif de découvrir et maîtriser les concepts fondamentaux de Docker et de l'orchestration de conteneurs à travers la mise en place d'une application multi-conteneurs.

Les objectifs principaux sont :

Comprendre le fonctionnement des conteneurs Docker
Créer une image personnalisée avec Dockerfile
Optimiser le build avec .dockerignore
Configurer une base de données PostgreSQL
Utiliser Docker Compose pour orchestrer plusieurs services
Connecter les conteneurs via un réseau Docker
Utiliser pgAdmin pour administrer PostgreSQL
Mettre en place un volume persistant pour la base de données

A.U : 2025/2026

Étape 1 : Préparation de l'application

1.1 Création de la structure du projet

On commence par créer l'arborescence du projet :

```
Last login: Thu May  7 15:49:57 on ttys000
macbook@MacBook-Pro ~ % noode --version
zsh: command not found: noode
macbook@MacBook-Pro ~ % node --version
v20.6.0
macbook@MacBook-Pro ~ % docker --version
Docker version 29.2.1, build a5c7197d72
macbook@MacBook-Pro ~ % mkdir project
cd project

mkdir frontend backend
touch docker-compose.yml
macbook@MacBook-Pro project % cd frontend
touch index.html
macbook@MacBook-Pro frontend % echo "<!DOCTYPE html>
<html>
<head>
  <title>Docker App</title>
</head>
<body>
  <h1>Bienvenue dans l'application Dockerisée</h1>
</body>
</html>">index.html
zsh: event not found: DOCTYPE
macbook@MacBook-Pro frontend % nano index.html
macbook@MacBook-Pro frontend % cat index.html
<!DOCTYPE html>
<html>
<head>
  <title>Docker App</title>
</head>
<body>
  <h1>Bienvenue dans l'application Dockerisée</h1>
</body>
</html>
macbook@MacBook-Pro frontend % touch Dockerfile
macbook@MacBook-Pro frontend % nano Dockerfile
macbook@MacBook-Pro frontend % touch .dockerignore
macbook@MacBook-Pro frontend % nano .dockerignore
macbook@MacBook-Pro frontend % cat .dockerignore
*.log

macbook@MacBook-Pro frontend % cat Dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
macbook@MacBook-Pro frontend % cd ../backend
touch db.env
macbook@MacBook-Pro backend % nano db.env
macbook@MacBook-Pro backend % cat db.env
POSTGRES_USER=admin
POSTGRES_PASSWORD=adminpassword
POSTGRES_DB=myapp
macbook@MacBook-Pro backend % cd ..
touch docker-compose.yml
macbook@MacBook-Pro project % nano docker-compose.yml
```

```
macbook@MacBook-Pro project % tree
.
├── backend
│   └── db.env
├── docker-compose.yml
└── frontend
    ├── Dockerfile
    └── index.html

3 directories, 4 files
```

1.2 Création du fichier HTML

Dans le dossier frontend/, on crée le fichier index.html :

```
TP3-Docker > src > DockerTp > frontend > index.html > html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <h1>Bienvenue dans l'application Dockerisée</h1>
</body>
</html>
```

Ce fichier représente la partie frontend de notre application.

Étape 2 : Création du Dockerfile

2.1 Configuration du serveur Nginx

Dans frontend/Dockerfile, on ajoute :

```
TP3-Docker > src > DockerTp > frontend > Dockerfile > ...

FROM nginx:alpine (last pushed 2 weeks ago)    The image
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

FROM nginx:alpine : utilise une image légère de Nginx
COPY : copie le fichier HTML dans le serveur web
EXPOSE 80 : expose le port 80
CMD : démarre Nginx

2.2 Création du fichier .dockerignore

Dans frontend/.dockerignore :

```
TP3-Docker > src > DockerTp > frontend > .dockerignore
*.log
```

*.log

Ce fichier permet d'ignorer les fichiers inutiles lors du build Docker afin d'optimiser la taille de l'image.

Étape 3 : Configuration de PostgreSQL

3.1 Variables d'environnement

Dans backend/db.env :

TP3-Docker > src > DockerTp > backend > .ENV db.env

```
1  POSTGRES_USER=admin
2  POSTGRES_PASSWORD=adminpassword
3  POSTGRES_DB=myapp
4
```

POSTGRES_USER=admin
POSTGRES_PASSWORD=adminpassword
POSTGRES_DB=myapp

Ces variables permettent de configurer automatiquement :

l'utilisateur PostgreSQL
le mot de passe
la base de données

Étape 4 : Création du fichier docker-compose.yml

Dans docker-compose.yml, on configure les différents services :

TP3-Docker > src > DockerTp > docker-compose.yml > ...

```yaml
version: "3.8"
services:
  frontend:
    build:
      context: ./frontend
    ports:
      - "8080:80"
    networks:
      - app-network
  db:
    image: postgres:13
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    env_file:
      - ./backend/db.env
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-network
  pgadmin:
    image: dpage/pgadmin4
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@admin.com
      PGADMIN_DEFAULT_PASSWORD: admin
    ports:
      - "5050:80"
    depends_on:
      - db
    networks:
      - app-network
volumes:
  db-data:
networks:
  app-network:
```

frontend
db
pgadmin
db-data
app-network

Explication des services

Frontend

Le service frontend construit une image Docker personnalisée basée sur Nginx et expose le port 8080.

Base de données PostgreSQL

Le service db utilise l'image officielle PostgreSQL 13 et stocke les données dans un volume persistant.

pgAdmin

Le service pgadmin permet d'administrer PostgreSQL via une interface web accessible depuis le navigateur.

Étape 5 : Test et validation

5.1 Construction et démarrage des conteneurs

Exécution de la commande : docker-compose up --build

```
macbook@MacBook-Pro project % docker-compose up --build
WARN[0000] /Users/macbook/project/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avo
id potential confusion
WARN[0000] Docker Compose requires buildx plugin to be installed
Sending build context to Docker daemon     361B
Step 1/5 : FROM nginx:alpine
 ---> 5616878291a2
Step 2/5 : COPY index.html /usr/share/nginx/html/index.html
 ---> Using cache
 ---> 27bd0c204354
Step 3/5 : EXPOSE 80
 ---> Using cache
 ---> 049f9516ab54
Step 4/5 : CMD ["nginx", "-g", "daemon off;"]
 ---> Using cache
 ---> b121783705b7
Step 5/5 : LABEL com.docker.compose.image.builder=classic
 ---> Using cache
 ---> a77ff569027e
Successfully built a77ff569027e
Successfully tagged project-frontend:latest
[+] up 1/1
 ✔ Image project-frontend Built                                  0.4s
Attaching to db-1, frontend-1, pgadmin-1
frontend-1  | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
frontend-1  | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
frontend-1  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
frontend-1  | 10-listen-on-ipv6-by-default.sh: info: IPv6 listen already enabled
frontend-1  | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
frontend-1  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
frontend-1  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
frontend-1  | /docker-entrypoint.sh: Configuration complete; ready for start up
frontend-1  | 2026/05/07 19:30:12 [notice] 1#1: using the "epoll" event method
frontend-1  | 2026/05/07 19:30:12 [notice] 1#1: nginx/1.29.8
frontend-1  | 2026/05/07 19:30:12 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
frontend-1  | 2026/05/07 19:30:12 [notice] 1#1: OS: Linux 6.8.0-90-generic
frontend-1  | 2026/05/07 19:30:12 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
frontend-1  | 2026/05/07 19:30:12 [notice] 1#1: start worker processes
frontend-1  | 2026/05/07 19:30:12 [notice] 1#1: start worker process 22
frontend-1  | 2026/05/07 19:30:12 [notice] 1#1: start worker process 23
db-1        |
db-1        | PostgreSQL Database directory appears to contain a database; Skipping initialization
db-1        |
db-1        | 2026-05-07 19:30:13.165 UTC [1] LOG:  starting PostgreSQL 13.23 (Debian 13.23-1.pgdg13+1) on x86_64-pc-linux-gnu, compil
ed by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
db-1        | 2026-05-07 19:30:13.165 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
db-1        | 2026-05-07 19:30:13.165 UTC [1] LOG:  listening on IPv6 address "::", port 5432
db-1        | 2026-05-07 19:30:13.173 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1        | 2026-05-07 19:30:13.184 UTC [25] LOG:  database system was shut down at 2026-05-07 19:01:47 UTC
db-1        | 2026-05-07 19:30:13.220 UTC [1] LOG:  database system is ready to accept connections
pgadmin-1   | postfix/postlog: starting the Postfix mail system


w Enable Watch   d Detach
```

```
pgadmin-1   | /venv/lib/python3.14/site-packages/sshtunnel.py:1040: SyntaxWarning: 'return' in a 'finally' block
pgadmin-1   |   return (ssh_host,
pgadmin-1   | NOTE: Configuring authentication for SERVER mode.
pgadmin-1   |
pgadmin-1   | pgAdmin 4 - Application Initialisation
pgadmin-1   | ======================================
pgadmin-1   |
pgadmin-1   | postfix/postlog: starting the Postfix mail system
frontend-1  | 172.20.0.1 - - [07/May/2026:19:42:22 +0000] "GET / HTTP/1.1" 200 141 "-" "curl/7.86.0" "-"
pgadmin-1   | [2026-05-07 19:42:26 +0000] [1] [INFO] Starting gunicorn 23.0.0
pgadmin-1   | [2026-05-07 19:42:26 +0000] [1] [INFO] Listening at: http://[::]:80 (1)
pgadmin-1   | [2026-05-07 19:42:26 +0000] [1] [INFO] Using worker: gthread
pgadmin-1   | [2026-05-07 19:42:26 +0000] [123] [INFO] Booting worker with pid: 123
frontend-1  | 172.20.0.1 - - [07/May/2026:19:42:27 +0000] "GET / HTTP/1.1" 200 141 "-" "curl/7.86.0" "-"
pgadmin-1   | /venv/lib/python3.14/site-packages/sshtunnel.py:1040: SyntaxWarning: 'return' in a 'finally' block
pgadmin-1   |   return (ssh_host,
frontend-1  | 172.20.0.1 - - [07/May/2026:19:42:51 +0000] "GET / HTTP/1.1" 200 141 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7
) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
frontend-1  | 2026/05/07 19:42:52 [error] 31#31: *4 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory),
client: 172.20.0.1, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "localhost:8090", referrer: "http://localhost:8090/
"
frontend-1  | 172.20.0.1 - - [07/May/2026:19:42:52 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://localhost:8090/" "Mozilla/5.0 (M
acintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
frontend-1  | 172.20.0.1 - - [07/May/2026:19:44:07 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
frontend-1  | 172.20.0.1 - - [07/May/2026:19:44:08 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
frontend-1  | 172.20.0.1 - - [07/May/2026:19:44:08 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
frontend-1  | 172.20.0.1 - - [07/May/2026:19:44:08 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
```

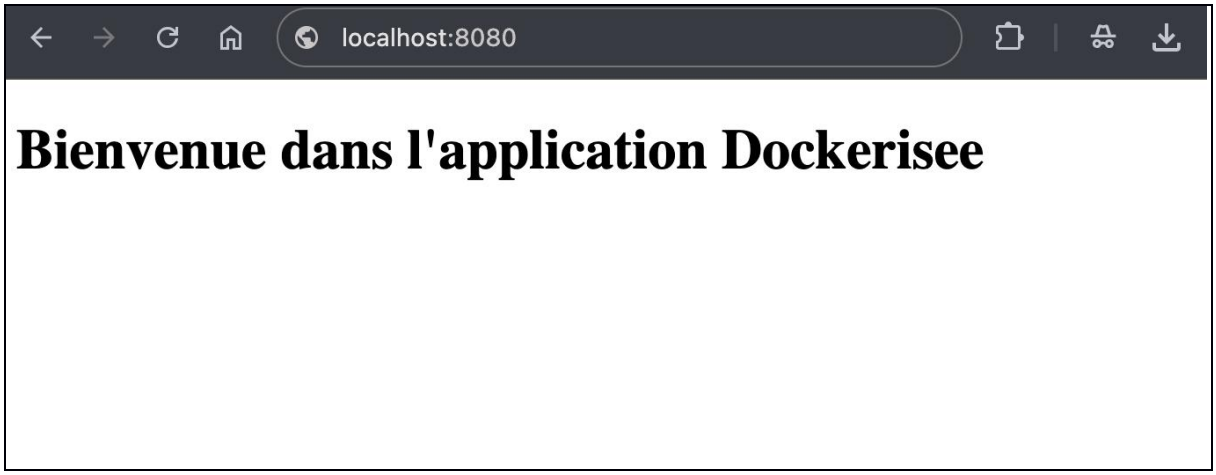
Résultat obtenu :
Successfully built project-frontend
Container project-db-1 Started
Container project-frontend-1 Started
Container project-pgadmin-1 Started

Les logs montrent que :

le frontend Nginx fonctionne correctement
PostgreSQL est prêt à accepter les connexions
pgAdmin a démarré avec succès

5.2 Vérification du frontend

On accède au frontend via :

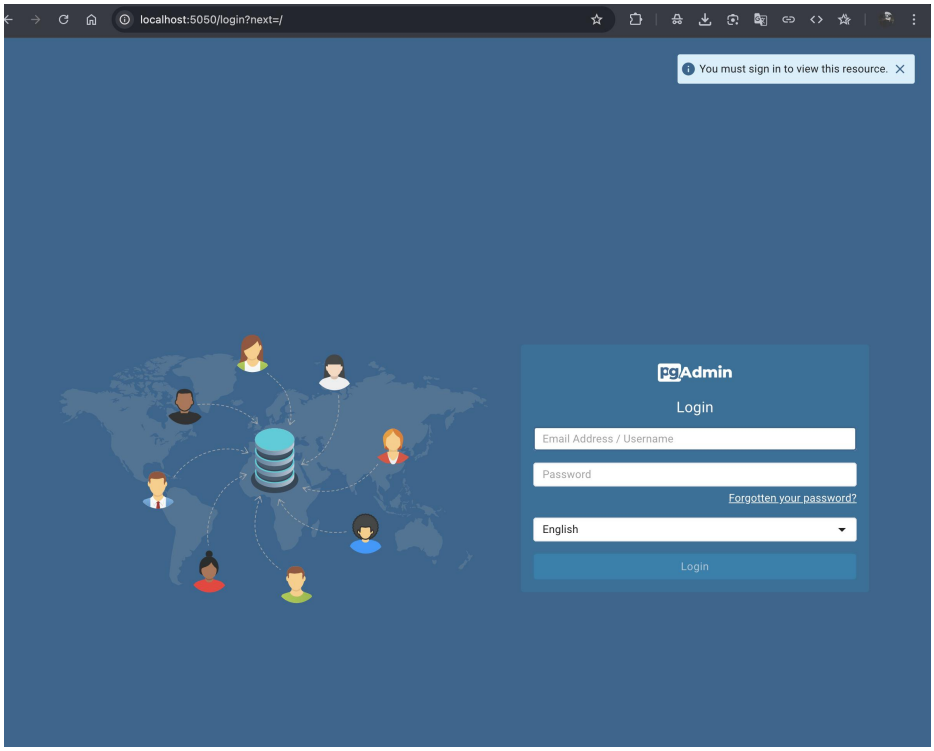


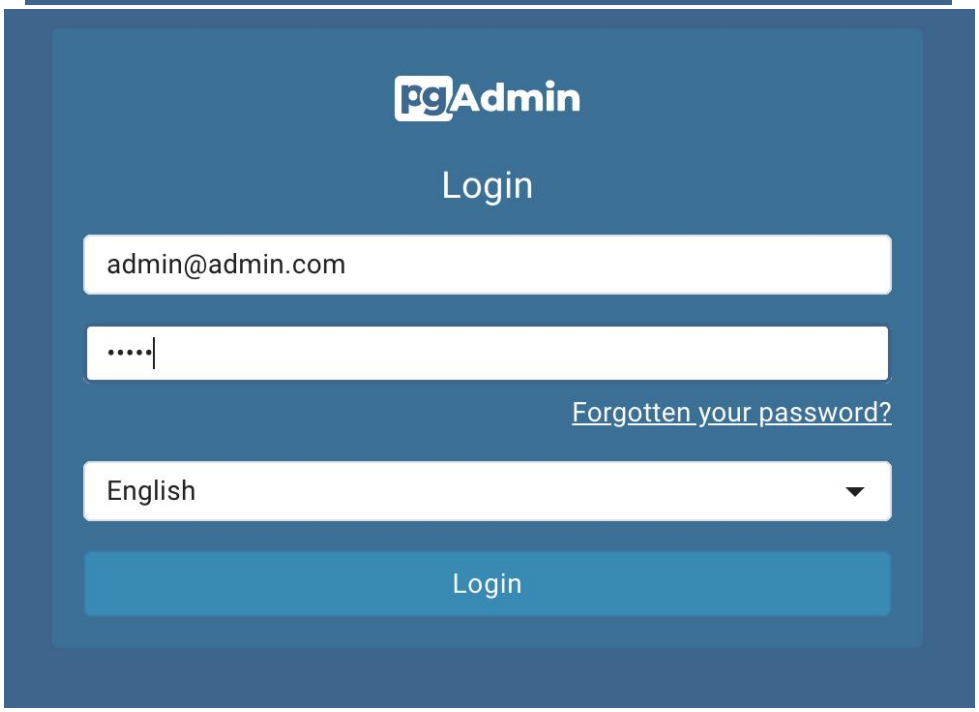
http://localhost:8080

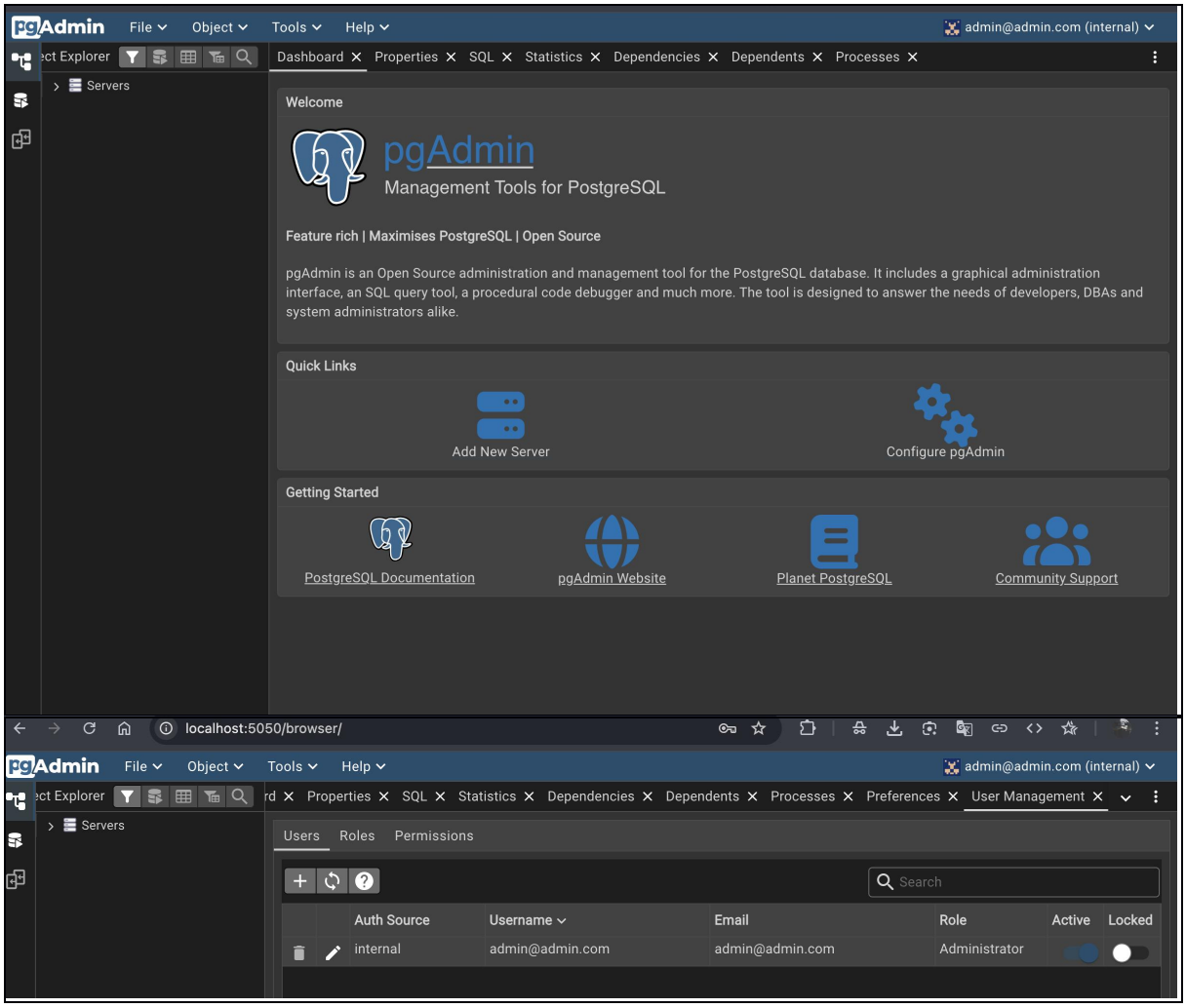
Résultat obtenu :
Bienvenue dans l'application Dockerisée
Le serveur Nginx affiche correctement la page HTML.

5.3 Connexion à pgAdmin

Accès à pgAdmin :
http://localhost:5050
Identifiants :
Email : admin@admin.com
Mot de passe : admin







La connexion à l'interface pgAdmin a été effectuée avec succès.

5.4 Configuration de la connexion PostgreSQL

Dans pgAdmin, on ajoute un nouveau serveur avec les paramètres suivants :

Param è tre	Valeur
Host	db
Port	5432
Username	admin
Password	adminpassword

La connexion à PostgreSQL fonctionne correctement.

5.5 Volume persistant

- Un volume Docker a été configuré :

db-data:/var/lib/postgresql/data

Ce volume permet de conserver les données même après l'arrêt ou le redémarrage des conteneurs.

- Test effectué :

docker-compose down
docker-compose up -d

- Résultat :

Les données de la base PostgreSQL restent sauvegardées après redémarrage.

Conclusion

Ce TP nous a permis de mettre en pratique les notions fondamentales de Docker et de l'orchestration de services multi-conteneurs.

Nous avons appris à :

- ✓ créer une image Docker personnalisée avec Dockerfile
- ✓ configurer plusieurs services avec Docker Compose
- ✓ connecter les conteneurs via un réseau Docker
- ✓ gérer une base de données PostgreSQL
- ✓ utiliser pgAdmin pour l'administration de la base
- ✓ mettre en place un volume persistant

Ce TP nous a également permis de comprendre l'importance de la conteneurisation dans le déploiement moderne des applications.